

Node.js

Complete Study Notes

Runtime · V8 Engine · CJS Modules · ES Modules

■ Advanced Web Programming

■ GTU — Semester VI

■ Subject: 3161611

■ Table of Contents

1	What is Node.js?	Introduction & overview of Node.js runtime
2	V8 JavaScript Engine	How V8 works, compilation pipeline, AST
3	How C++ Understands JS	Parser, Compiler, AST explained in detail
4	Why C++ for Node.js?	Performance, embedding, memory management
5	CommonJS Module System (CJS)	require(), module.exports, IIFE wrapping
6	Multiple Exports in CJS	Exporting objects, named exports
7	ES Modules (ESM)	import/export syntax, .mjs, package.json
8	CJS vs ESM Comparison	Side-by-side feature comparison table
9	Quick Reference & Summary	Key concepts, commands, cheat sheet

1. What is Node.js?

Definition

Node.js is an **open-source, cross-platform JavaScript runtime environment** that allows developers to execute JavaScript code **outside of a web browser**. It is built on **Chrome's V8 JavaScript engine** and extends JS capabilities for server-side development.

Key Characteristics

Open-Source	Free to use, modify, and distribute. Large community support.
Cross-Platform	Runs on Windows, macOS, Linux — write once, run anywhere.
Non-Blocking I/O	Uses event-driven, asynchronous architecture for high performance.
Single-Threaded	Runs on a single thread but handles concurrency via the event loop.
NPM Ecosystem	Access to 1.5 million+ packages via Node Package Manager (npm).
Server-Side JS	Enables JavaScript to run on servers, not just in browsers.

Extra Functionalities in Node.js

Node.js provides additional **global objects and methods** that are not part of the standard JavaScript language but are built into Node.js to facilitate server-side development. Examples include:

- **process** — Access environment variables, command-line args, exit codes.
- **__dirname / __filename** — Absolute path of the current directory/file.
- **Buffer** — Handle binary data streams (TCP streams, file I/O).
- **require() / module.exports** — CommonJS module system.
- **fs, http, path, os** — Built-in core modules for system access.
- **setTimeout / setInterval** — Timer functions (same as browser, but server-side).

■ *Node.js is NOT a programming language — it is a RUNTIME ENVIRONMENT. The language is still JavaScript!*

2. V8 JavaScript Engine

What is V8?

V8 is the **JavaScript engine** developed by Google, written in **C++**. It is the engine that powers Google Chrome and is the core of Node.js. V8's primary role is to **parse and compile JavaScript** into native machine code so the CPU can execute it at maximum speed.

V8 Execution Flow

JS Source Code → V8 Engine (C++) → Bytecode (Ignition) → Machine Code (TurboFan) → CPU Execution

ECMAScript Compliance

V8 follows the **ECMAScript (ECMA-262) specifications** to understand and interpret JavaScript code. This ensures that JavaScript behaves consistently across all environments that use V8 (Chrome, Node.js, Deno, etc.).

V8 Internal Components

Component		Role
Parser		Reads JS source code and converts it into an Abstract Syntax Tree (AST)
Ignition		Interpreter that converts AST to bytecode for fast startup
TurboFan		Optimizing compiler — converts hot bytecode to optimized machine code
Garbage Collector (Orinoco)		Automatic memory management — frees unused objects
Inline Cache		Optimizes repeated property lookups for speed

JIT Compilation

V8 uses **Just-In-Time (JIT) Compilation**. Instead of purely interpreting JS (slow) or pre-compiling everything (slow startup), V8 interprets first via Ignition and then optimizes hot (frequently executed) code paths using TurboFan at runtime.

■ TIP: V8 deoptimizes code if types change unexpectedly. Always use consistent types for function arguments to keep code fast!

3. How C++ Understands JavaScript

The Two-Phase Process

The V8 engine (written in C++) understands JavaScript through a **two-phase process**: parsing and compiling.

Phase 1: Parsing → Abstract Syntax Tree (AST)

The **Parser** reads the JavaScript source code character by character and converts it into a structured data representation called the **Abstract Syntax Tree (AST)**.

What is an AST?

- A **tree-shaped data structure** that represents the grammatical structure of your code.
- Every statement, expression, and declaration becomes a node in this tree.
- The AST strips away whitespace and comments — keeping only the meaningful structure.
- This intermediate form makes it easy for the compiler to analyze and optimize code.

AST Example — JS Code vs Tree Representation

```
// JavaScript source:
function sum(a, b) {
  return a + b;
}

// AST representation (simplified):
FunctionDeclaration {
  id: Identifier { name: 'sum' }
  params: [ Identifier{a}, Identifier{b} ]
  body: BlockStatement {
    body: [ ReturnStatement {
      argument: BinaryExpression {
        operator: '+',
        left: Identifier { name: 'a' }
        right: Identifier { name: 'b' }
      }
    }
  ]
}
```

Phase 2: Compilation → Machine Code

The **Compiler** (Ignition + TurboFan) takes the AST and translates it into **machine code** — binary instructions that the computer's CPU can execute directly. This is why JavaScript in V8 runs extremely fast.

JS Source → Lexer/Tokenizer → Parser → AST → Ignition (Bytecode) →
TurboFan (Machine Code) → CPU

4. Why C++ for Node.js?

Historical Context

Web servers were **already built in C++** before Node.js existed. The server infrastructure, operating system interfaces, and low-level networking stacks were all written in C/C++. By writing V8 in C++, it could be seamlessly embedded into existing server software.

Key Reasons C++ Was Chosen

■ High Performance	C++ is a compiled language with zero garbage-collection pauses. It gives fine-grained control over CPU and memory, crucial for a fast runtime.
■ Low-Level System Access	C++ can directly interface with the OS — file system, network sockets, processes — which are all needed for a server-side runtime.
■ Embeddability	V8 was designed to be embedded into any C++ application. Node.js is literally a C++ application with V8 embedded inside it.
■ Existing Infrastructure	Servers already understood C++ code. Embedding V8 meant servers could now also run JavaScript without any architectural overhaul.
■ Memory Management	Unlike Java or Python (which have automatic GC), C++ allows manual memory management, making it faster for performance-critical operations.
■ Cross-Platform	C++ can be compiled for any platform (Linux, Windows, macOS), making Node.js truly cross-platform from the ground up.

■ Key Insight: V8 can be embedded into ANY C++ application — not just Node.js. This is why you can also find V8 in MongoDB, Electron, and other tools!

5. CommonJS Module System (CJS)

What is a Module?

A **module** is a reusable piece of code isolated in its own file. Node.js uses modules to organize code, prevent naming conflicts, and promote reusability. **CommonJS (CJS)** is the default module system in Node.js, using **require()** to import and **module.exports** to export.

5.1 Basic require() — Importing a File

When you use **require('./filename')**, Node.js loads and executes that file. However, **functions and variables defined in that file are private by default**.

```
// second.js
console.log('welcome from 2nd file');

function sum(a, b) {
  return a + b;
}

// sum is NOT exported – it is private to this file

// first.js
require('./second');           // ■ executes second.js
const result = sum(2, 3);      // ■ ERROR: sum is not defined
console.log(result);
```

■ *Why ERROR? Because require() wraps each file in an IIFE, making all variables/functions local to that scope.*

5.2 The IIFE Wrapping — Behind the Scenes

When Node.js loads a module using **require()**, it **wraps the entire file's code in an Immediately Invoked Function Expression (IIFE)**. This is why everything is private by default:


```
// What Node.js actually does behind the scenes:
(function(exports, require, module, __filename, __dirname) {

    // ← Your second.js code runs HERE inside this function
    console.log('welcome from 2nd file');

    function sum(a, b) {
        return a + b;
    }

    // sum is scoped inside this IIFE – not accessible outside

})();

// Back in first.js:
const result = sum(2, 3);    // ■ sum is not in this scope!
```

IIFE Benefits

- ✓ **Encapsulation** — Each module has its own private scope.
- ✓ **No Global Pollution** — Variables don't leak into the global scope.
- ✓ **Security** — Internal implementation details are hidden.
- ✓ **Dependency Management** — Each module manages its own imports.

5. CJS — module.exports & require()

5.3 Exporting a Single Function

To make a function or variable available to other files, use **module.exports**. Whatever you assign to `module.exports` is what `require()` returns in the importing file:

```
// second.js — EXPORTING
console.log('welcome from 2nd file');

function sum(a, b) {
  return a + b;
}

module.exports = sum;    // ■ Export the sum function
```

```
// first.js — IMPORTING
const sum = require('./second');    // sum now holds the function

console.log(sum(2, 6));              // Output: 8
console.log('welcome to backend');
```

5.4 Exporting Multiple Functions (Object Syntax)

```
// second.js
function sum(a, b)      { return a + b; }
function multiply(a, b) { return a * b; }

module.exports = { sum, multiply };    // export as an object
```

```
// first.js — destructured import
const { sum, multiply } = require('./second');

console.log(sum(2, 6));              // 8
console.log(multiply(2, 6));         // 12
```

5.5 Property-Based Export (exports shorthand)

module.exports is just an empty object `{}` by default. You can add properties to it directly:

```
// second.js – adding properties to module.exports  
function sum(a, b)      { return a + b; }  
function multiply(a, b) { return a * b; }  
  
module.exports.sum      = sum;          // attach sum  
module.exports.multiply = multiply;     // attach multiply
```

module.exports Deep Dive

Syntax	What it does
module.exports = fn	Replaces entire exports with fn
module.exports = { a, b }	Replaces with an object containing a and b
module.exports.x = val	Adds property x to the default empty object
exports.x = val	Shorthand — same as module.exports.x = val
exports = { ... }	■ BREAKS the reference — do NOT do this!

■ **REMEMBER:** *module.exports starts as an empty object {}. require() always returns whatever module.exports points to at the end of the file.*

7. ES Modules (ESM) — The Modern Way

Overview

ES Modules (ESM) is the **official JavaScript module standard** introduced in ES6 (ES2015). It uses **import** and **export** keywords instead of `require()` and `module.exports`. While browsers natively support ESM, **Node.js requires additional configuration** to use it.

7.1 Enabling ESM in Node.js

Method 1: File Extension

Rename your file from `.js` to `.mjs`
Node.js treats `.mjs` files as ES modules automatically.

Method 2: package.json

Add `"type": "module"` to your `package.json` file.
All `.js` files in the project will be treated as ESM.

```
// package.json – Enable ESM for all .js files
{
  "name": "my-app",
  "version": "1.0.0",
  "type": "module"    // ← Add this line
}
```

7.2 Named Exports with ESM

```
// second.js – Named export
console.log('good to see you');

export function sum(a, b) {
  return a + b;
}

export function multiply(a, b) {
  return a * b;
}
```

```
// first.js – Named import (must include file extension in ESM!)
import { sum, multiply } from './second.js';

console.log(sum(5, 7));          // 12
console.log(multiply(3, 4));    // 12
```

7.3 Default Exports with ESM

```
// mathUtils.js – Default export
export default function greet(name) {
  return `Hello, ${name}!`;
}

// main.js – Default import (no curly braces)
import greet from './mathUtils.js';
console.log(greet('Node'));    // Hello, Node!
```

8. CJS vs ESM — Side-by-Side Comparison

Understanding the differences between CommonJS and ES Modules is crucial for modern Node.js development:

Feature	CommonJS (CJS)	ES Modules (ESM)
Syntax (import)	<code>const x = require('./file')</code>	<code>import x from './file.js'</code>
Syntax (export)	<code>module.exports = x</code>	<code>export default x</code>
Named exports	<code>module.exports = { a, b }</code>	<code>export { a, b }</code>
File extension	.js (default)	.mjs or type:module
Loading	Synchronous	Asynchronous
When evaluated	Runtime (dynamic)	Parse time (static)
Top-level await	■ Not supported	■ Supported
Tree shaking	■ Not supported	■ Supported
__dirname available	■ Yes	■ Use import.meta.url
Circular deps	Partial support	Better handled
Browser compatible	■ No (Node only)	■ Yes (native in browsers)

■ **Best Practice:** For new Node.js projects, prefer ESM (ES Modules) as it is the modern standard and supports tree shaking, top-level await, and browser compatibility.

9. Quick Reference & Summary

Core Concepts Cheat Sheet

Node.js	JS runtime built on V8; runs JS outside the browser on the server.
V8 Engine	Google's C++ JS engine; converts JS → AST → bytecode → machine code.
AST	Abstract Syntax Tree — tree structure representing JS code grammar.
JIT	Just-In-Time compilation — V8 optimizes hot code paths at runtime.
CJS	CommonJS — Node.js default module system using require/module.exports.
ESM	ES Modules — modern standard using import/export; needs config in Node.
IIFE	Immediately Invoked Function Expression — Node wraps each CJS module in one.
module.exports	The object CJS uses to expose values from a module (default: {}).
require()	CJS function to synchronously load and execute another module file.
import	ESM keyword to statically import bindings from another module.
export	ESM keyword to expose bindings from a module.
.mjs	File extension to use ESM in Node.js without package.json config.

Important Rules to Remember

- ★ Node.js is a **runtime** — not a language or framework.
- ◆ V8 follows **ECMAScript specs** — same JS rules everywhere.
- ★ CJS uses **synchronous** loading — ESM uses **asynchronous** static analysis.
- ◆ Every CJS file is wrapped in an **IIFE** — everything is private by default.
- ★ **module.exports = {}** by default — require() returns this object.
- ◆ In ESM, **always include the .js extension** in import paths.
- ★ **__dirname** works in CJS but NOT in ESM — use import.meta.url instead.
- ◆ ESM supports **top-level await**; CJS does not.
- ★ V8 **de-optimizes** code if variable types change — keep types consistent!